
Pusan National University

Computer Science and Engineering

Technical Report 2026-09

쿠버네티스 환경에서의 PaC위반 대응 및 예외 관리 시스템 설계



202355702 고영림

202155559 박태영

202148124 최유렬

지도교수 유영환

목차

1	서론	1
1.1	연구 배경	1
1.2	기존 문제점	1
1.3	연구 목표	2
2	연구 배경	3
2.1	Kubernetes 정책 집행과 예외 리소스	3
2.2	GitOps 및 MLOps와의 연계	3
2.3	기존 연구와의 차별성	4
3	연구 내용	5
3.1	요구사항과 설계 변경	5
3.1.1	기능 요구사항	5
3.1.2	제약사항과 대응	6
3.2	시스템 구조와 데이터 설계	6
3.3	핵심 기능 구현	8
3.3.1	인증과 접근 제어	8
3.3.2	위반 조회와 원인 설명	8
3.3.3	예외 요청과 생명주기 관리	8
3.3.4	GitOps와 운영 가시화	10
3.3.5	MLOps 적용 기능	10
3.4	시스템 설계	11
4	연구 결과 분석 및 평가	15
4.1	평가 범위와 근거	15
4.2	핵심 기능 검증 시나리오	15
4.3	성능평가 설계와 결과 입력	17

4.4	시스템 결과물	21
4.5	멘토 의견 반영	24
4.6	현재 한계	24
5	구성원별 역할과 개발 일정	26
6	결론 및 향후 연구 방향	28
7.	참고문헌	29

1. 서론

1.1 연구 배경

Kubernetes 환경에서는 컨테이너 이미지, 실행 권한, 자원 요청량 등 배포 설정이 서비스의 안전성과 운영 효율에 영향을 준다. Policy-as-Code(PaC)는 이러한 기준을 코드로 정의하여 일관되게 검사하는 방식이다. Kubernetes의 어드미션 제어(Admission Control)는 리소스가 저장되기 전 요청을 검토하며, 정책 엔진은 이 과정에서 허용 조건을 검사하여 규정을 위반하는 리소스가 클러스터에 배포되는 것을 차단할 수 있다[1].

그러나 정책을 집행하는 것만으로 개발자의 대응 과정이 완성되지는 않는다. 배포가 거절되면 개발자는 위반 정책과 대상 리소스를 식별하고, 요청을 수정하거나 긴급 배포 또는 조직의 기술적 제약에 따른 한시적 예외를 요청해야 한다. 본 과제는 이러한 후속 대응 과정을 웹 기반 시스템으로 연결하고, 승인 이후 클러스터 내 예외의 적용·만료·감사 이력까지 통합 관리하는 거버넌스 플랫폼 구축을 목표로 한다.

1.2 기존 문제점

첫째, 정책 검증 메시지와 정책 YAML, 실제 리소스 상태가 서로 다른 위치에 존재하여 원인 파악에 시간이 소요된다. 둘째, 메신저나 수동 파일 변경으로 예외를 처리하면 요청 사유, 승인자, 적용 범위, 종료 시점을 일관되게 추적하기 어렵다. 셋째, 관리자 승인 기록과 실제 클러스터 반영 결과가 다를 수 있다. 네트워크 장애나 API 오류가 발생했음에도 승인 여부만 표시하면 개발자는 예외가 적용되었다고 오해할 수 있다.

특히 임시 예외를 생성한 뒤 제거를 누락하면 정책 우회가 장기간 유지된다. 따라서 본 시스템은 승인 결정과 실제 적용을 구분하고, DB의 업무 상태와 Kubernetes 리소스를 반복적으로 대조하여 적용 및 삭제를 재시도하도록 설계하였다. 정책 위반을 단순히 시각화하는 것을 넘어 대응의 책임과 종료 조건을 관리하는 데 과제의 초점을 두었다.

1.3 연구 목표

본 과제의 목표는 Kubernetes 정책 위반 이후의 대응 과정을 하나의 웹 기반 흐름으로 연결하는 것이다. 위반 정보 조회, 원인 설명, 예외 요청, 관리자 검토, PolicyException 적용, 만료 및 감사 이력 보존을 단절 없이 제공하여, 개발자와 관리자가 여러 도구를 오가지 않고 대응을 완료하도록 한다[2].

세부 목표는 다음과 같다. 첫째, 위반 메시지와 정책, 리소스 상태를 함께 제시하여 원인 파악에 드는 시간을 줄인다. 둘째, 승인 결정과 실제 클러스터 적용을 분리하고, 업무 상태와 Kubernetes 리소스를 반복 대조하며 적용·삭제를 재시도하여 정합성을 확보한다. 셋째, 예외의 만료·취소와 감사 이력을 관리하여 임시 예외가 장기간 방치되는 것을 방지한다. 넷째, 사용자별 역할과 접근 가능한 클러스터를 구분하여 동일한 기능도 허용된 범위에서만 사용하도록 한다. 아울러 GitOps 게시와 MLOps(Kubeflow·KServe) 연계를 확장 목표로 포함한다.

2. 연구 배경

2.1 Kubernetes 정책 집행과 예외 리소스

Kubernetes의 admission 제어는 리소스가 저장되기 전에 요청을 검토하며, 정책 엔진은 이 과정에서 허용 조건을 검사한다. Kyverno는 정책을 Kubernetes 리소스로 표현하고 검증 결과를 PolicyReport와 ClusterPolicyReport로 제공하는 대표적인 Policy-as-Code 엔진이며, 정책 위반은 정책·규칙 이름과 대상 리소스의 종류·이름·네임스페이스로 식별된다.

운영 과정에서는 긴급 배포, 테스트, 마이그레이션 등으로 일시적인 정책 예외가 필요할 수 있다. Kyverno는 PolicyException 리소스로 특정 정책과 규칙에 대해 제한된 리소스의 예외를 표현하며, 예외 기능 활성화와 허용 네임스페이스 설정이 선행되어야 한다[3]. 그러나 예외 리소스 자체는 요청자·승인자·사유·만료와 같은 업무 맥락과 이력을 담지 않는다.

또한 admission에서 거절된 리소스는 클러스터에 저장되지 않을 수 있어, 위반 목록에 모든 배포 거절이 자동으로 남는다고 볼 수는 없다. 예외가 만료되어도 이미 실행 중인 워크로드가 자동으로 종료되지는 않으며, 예외 제거 이후의 후속 생성·변경 요청부터 정책이 다시 적용된다. 이러한 특성은 위반 이후의 대응을 별도의 업무 흐름으로 관리할 필요성을 보여준다.

2.2 GitOps 및 MLOps와의 연계

정책과 예외를 선언적 구성으로 관리하면 변경 이력과 재현성을 확보할 수 있다. 이에 본 과제는 런타임 적용과 함께 예외 YAML 생성과 GitHub Pull Request 발행 경로를 제공하여, 예외를 Git 기반 형상 관리로 게시하는 GitOps 연계를 구성한다.

Kubernetes는 ML 워크로드의 실행 기반으로 널리 사용된다. 본 과제는 그 연장선에서 Kubeflow Notebook, KServe 모델 서빙, Kubeflow Pipelines를 정책 통제와 연결하는 모듈을 포함한다. Kubeflow Notebook은 Kubernetes 위에서 개발 환경을 실행하는 구성 요소로, CPU·메모리·이미지 설정을 정책으로 통제하기에 적합한 사례이다[4]. 본 과제에서는 ML 모델의 정확도 향상보다 ML 워크로드의 생성과 운영에 정책 기준을 적용하는 문제를

다룬다.

2.3 기존 연구와의 차별성

정책 위반을 시각화하는 대시보드나 정책 검사 도구는 이미 존재하지만, 대부분 위반의 가시화나 정책 적용 여부 확인에 초점을 두어 위반 이후의 대응 과정은 개발자의 수작업이나 메신저 협의에 의존한다. GitOps 도구는 선언적 구성의 일관성은 보장하지만 예외의 승인 주체·사유·기간과 같은 업무 맥락과 감사 이력을 다루지 않는다.

본 과제는 이러한 접근과 달리 승인 결정과 실제 클러스터 반영을 상태 모델에서 분리하고, 업무 상태와 Kubernetes 리소스를 반복 대조·재시도하여 정합성을 유지하며, 예외의 요청·승인·적용·만료·취소·감사를 하나의 생명주기로 통합한다. 또한 위반 조회, 원인 설명, GitOps 게시, MLOps 워크로드에 대한 정책 통제를 같은 흐름에 연결한다는 점에서 차별성을 갖는다.

3. 연구 내용

3.1 요구사항과 설계 변경

착수 단계에서는 위반 메시지 해석과 정책 예외 처리를 함께 주요 기능으로 정의하였다. 중간 단계에서는 예외 처리 전체 흐름을 먼저 검증하기 위해 인증·인가와 거버넌스 기능을 우선하였고, 메시지 보강은 후속으로 분리하였다. 이후 실제 위반 조회, 예외 생명주기, 오류 설명 및 MLOps 적용으로 구현 범위를 확장하였다. 요구사항과 최종 구현·검증 범위의 비교는 표 3.1와 같다.

표 3.1: 기존 요구사항과 최종 구현·검증 범위 비교

항목	기존 요구사항	현재 구현 범위
위반 대응	위반 가시화와 메시지 보강 계획	PolicyReport 조회, DB 이력, 처리 상태 변경, AI·규칙 기반 설명
예외 관리	요청·승인·만료 흐름 설계	중간 상태와 재시도, 취소, 실패 복구, 감사 기록 구현
접근 제어	개발자·관리자 역할 구분	JWT, 역할별 Permission, 사용자별 클러스터 할당
GitOps	Git 기반 변경 이력 유지	YAML 및 PR 생성 경로 구현
적용 사례	일반 Kubernetes 리소스	Notebook·서빙·파이프라인 연동 모듈 추가

3.1.1 기능 요구사항

개발자는 허용된 클러스터의 정책과 위반을 조회하고, 대상 정책·규칙·리소스 및 만료 시각을 지정하여 예외를 요청한다. 승인자는 요청 사유와 적용 범위를 검토하여 승인하거나

거절한다. 요청 생성 시 미래의 만료 시각과 최대 허용 기간을 검증하며, 승인 시에는 정책과 규칙의 유효성을 다시 확인한다. 요청 규칙 중 일부만 승인하는 경우에도 최초 요청 범위를 임의로 확대하지 않도록 검사한다.

관리자는 사용자 역할과 클러스터 접근 범위를 관리하고, 감사 로그로 상태 변경을 추적한다. 예외 적용 실패는 재시도 횟수와 오류를 보존하며, 만료 또는 취소는 Kubernetes 예외 삭제와 연결한다. 원인 설명 기능은 조치 방향을 제공하되 승인 권한이나 정책 검증을 대신하지 않는다.

3.1.2 제약사항과 대응

개발 장비의 자원 제약과 팀원별 환경 차이에 대응하기 위해 DevContainer, pnpm workspace, kind 및 컨테이너 배포 구성을 사용하였다. 데이터베이스 변경과 Kubernetes API 호출은 단일 트랜잭션으로 묶을 수 없으므로 업무 상태를 먼저 기록하고 외부 적용 결과를 별도로 확인한다. DB 접근은 Prisma로 통일하고, 외부 시스템은 Kubernetes, Kubeflow, KServe, KFP 어댑터를 통해 연결한다.

정책 엔진 자체의 집행 경로에 본 서비스의 DB 저장이나 LLM 분석을 삽입하지 않도록 구성하였다. 따라서 보고서에는 대시보드 조회 지연, 예외 반영 지연, AI 설명 지연을 각각 측정하며 이를 admission 지연과 혼동하지 않는다. 멀티클러스터 구성도 지원 코드의 존재와 실제 여러 클러스터에서의 운영 검증을 구분하여 평가한다.

3.2 시스템 구조와 데이터 설계

시스템은 Next.js 프론트엔드, NestJS API, PostgreSQL과 Prisma, Kubernetes 및 Kyverno, 선택적 외부 연동으로 구성된다. 프론트엔드는 개발자·관리자 화면을 제공하고, 백엔드는 인증, 정책, 위반, 예외, 감사 및 MLOps 모듈을 결합한다. 배포 애플리케이션은 apps/frontend와 apps/backend로 분리하였다. 시스템 전체 구조는 그림 3.1과 같고, 구성 요소별 책임은 표 3.2에 정리하였다.

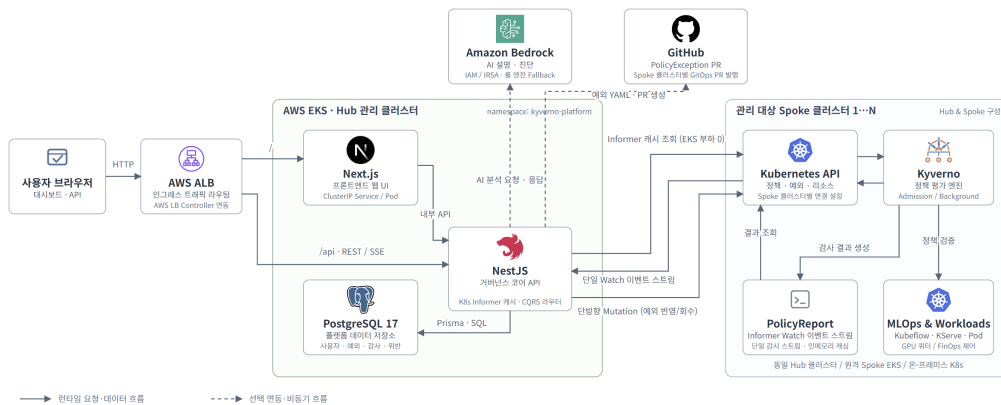


그림 3.1: 시스템 전체 구조

표 3.2: 시스템 구성 요소와 책임

구성 요소	책임	주요 연결
프론트엔드	목록·필터·상세·승인 화면 및 클러스터 현황	NestJS API, 일부 SSE와 조회 갱신
백엔드	권한 검사, 요청 검증, 상태 전이 및 감사 기록	Prisma, 외부 연동 어댑터
PostgreSQL	사용자·요청·위반 이력·감사 데이터 보존	Prisma 스키마와 마이그레이션
쿠버네티스 클러스터	정책과 보고서 조회, 예외 생성·대조·삭제	Kubernetes API Server
선택적 확장	설명 생성, GitOps 게시, ML 워크로드 제어	Bedrock, GitHub, Kubeflow·KServe·KFP

정책 집행 흐름은 “리소스 생성·변경 요청 → Kubernetes API → Kyverno 검사 → 허용 또는 거절”이다. 대응 흐름은 “위반 또는 거절 정보 확인 → 설명과 수정 방향 검토 → 예외 요청 → 승인자의 판단 → 실제 적용 확인 → 만료·취소 → 감사 이력 조회”로 구성된다. 두 흐름을 구분하면 정책 엔진이 판단하는 영역과 본 시스템이 지원하는 영역을 명확히 설명할 수 있다.

데이터 모델에서 PolicyExceptionRequest는 대상 클러스터, 정책·규칙, 리소스, 사유, 승인자, 만료 시각, 실제 활성화 시각, 재시도 정보를 보존한다. AuditLog는 행위자 유형과 전후 상태를 저장하며 사용자 행위와 시스템 행위를 구분한다. UserCluster는 사용자와 접근 가능한 클러스터 ID의 연결을 나타낸다. 클러스터 접속 구성 자체는 DB 엔티티 대신 설정과

ClusterProvider에서 관리한다. 요청·감사 등 주요 엔터티의 식별자는 시간 정렬성이 있는 UUIDv7을 사용한다.

ViolationHistory는 클러스터, 정책, 규칙, 리소스, 심각도, 처리 상태, 발생 시각을 저장한다. 실시간 보고서와 서비스의 업무 상태를 결합하되, 보고서 항목 순서가 바뀌었다는 이유로 사용자가 관리한 상태가 다른 리소스에 연결되지 않도록 식별 정보를 사용한다. 운영 이력은 Kubernetes 리소스의 삭제와 분리하여 보존하는 구조이다.

3.3 핵심 기능 구현

3.3.1 인증과 접근 제어

인증은 JWT Access Token과 Refresh Token으로 구성하였다. 프론트엔드는 Access Token을 메모리에서 관리하고, Refresh Token은 HttpOnly 쿠키를 통해 전달한다. 토큰 갱신 시 회전 및 폐기 처리를 수행하며, 사용자 비활성화와 세션 변경에 대응하는 경로를 마련하였다. 역할은 ADMIN, APPROVER, REQUESTER, VIEWER로 구분하고 역할별 Permission을 API에서 검사한다.

사용자별 클러스터 할당은 역할 검사와 별도로 적용한다. 예외 요청자는 자신의 요청을 중심으로 조회하고, 승인자의 자기 승인·거절은 차단하여 요청 주체와 검토 주체를 분리한다.

3.3.2 위반 조회와 원인 설명

위반 조회 서비스는 접근 가능한 클러스터의 PolicyReport와 ClusterPolicyReport에서 결과를 추출하고 정책, 네임스페이스, 심각도 등의 필터를 적용한다. 모든 대상 클러스터 조회가 실패하면 DB 이력 조회로 대체하고, 일부 클러스터만 실패하면 성공한 클러스터 결과를 반환하여 부분 장애에도 조회를 지속한다. 위반 이력 동기화는 30초 주기로 수행한다.

설명 API는 오류 메시지, 정책 YAML, 리소스 매니페스트와 환경 문맥을 입력받아 요약, 수정 단계, 수정 YAML, 정책의 필요성을 반환한다. Bedrock 응답 실패·시간 초과 또는 JSON 파싱 실패 시 규칙 기반 템플릿으로 전환한다. 현재 코드의 기본 대기 상한은 환경 설정으로 조정할 수 있다. 이는 관측된 평균 응답 시간이 아니라 구현 설정값이다.

LLM이 생성한 수정안은 응답 생성과 별개로 수정 YAML의 구문 유효성, 원래 위반 해결 여부, 불필요한 권한 확대 여부를 기준으로 평가한다.

3.3.3 예외 요청과 생명주기 관리

예외 상태는 PENDING, APPLYING, APPROVED, REJECTED, CANCELLING, EXPIRING, EXPIRED, CANCELLED, FAILED로 구성된다. 승인 결정은 PENDING에서

APPLYING으로 전환한 뒤 실제 PolicyException을 적용한다. 적용 성공을 확인한 경우 APPROVED로 전환하므로, 승인자의 판단과 인프라 반영 완료가 데이터 모델에서 구분된다. 전체 상태 전이는 표 3.3 및 그림 3.2과 같다.

표 3.3: 예외 상태 전이

상황	주요 전이	처리 의미
승인·적용	PENDING → APPLYING → APPROVED	승인 기록 후 리소스 적용을 확인
거절	PENDING → REJECTED	승인자 판단과 사유 보존
만료	APPLYING 또는 APPROVED → EXPIRING → EXPIRED	만료를 확인하고 예외 리소스 삭제
취소	PENDING → CANCELLED 또는 적용 상태 → CANCELLING → CANCELLED	미적용 요청 종료 또는 적용 리소스 회수
적용 실패	APPLYING → 재시도 → FAILED	실패 이력을 보존하고 수동 재시도 제공

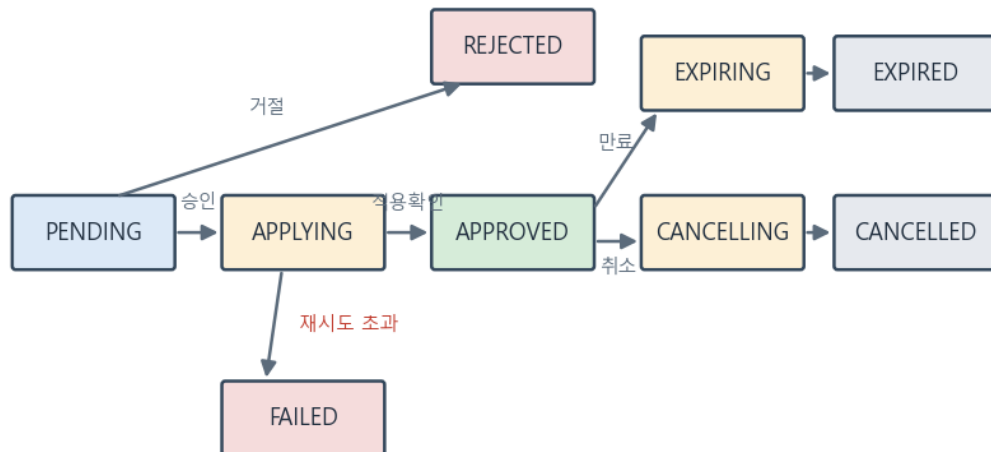


그림 3.2: 예외 요청 생명주기 상태 전이

조정 작업은 10초마다 최대 50건의 후보를 선택하고 클러스터별로 묶어 처리한다. 상태를 조건으로 한 DB 갱신과 감사 기록을 트랜잭션에서 수행하여 중복 승인이나 경쟁 상태를 다룬다. 적용 실패에는 10초 기반의 지수 백오프와 최대 300초 대기 간격을 사용하며, APPLYING

상태에서 10회 실패하면 FAILED로 전환한다. 삭제 실패는 삭제 중 상태를 유지하여 재시도한다. 이 수치는 처리량이나 만료 지연을 보장하는 성능 수치가 아니다.

승인 API와 자동 조정 작업이 동일한 예외를 중복 처리하지 않도록, 각 예외의 실행 소유권과 유효 기간을 데이터베이스에 기록하고 잠금 건너뛰기(FOR UPDATE SKIP LOCKED) 방식으로 처리 후보를 선점한다. 만료 작업을 우선 처리하고 정상 승인 건도 주기적으로 상태를 재확인하여, 동시 승인, 여러 조정 작업 간 경쟁, 적용 도중 취소, 실행 소유권 인계, 만료 중 중복 삭제를 방지한다.

PolicyException 매니페스트는 요청 ID와 관리 주체 라벨을 포함한다. Deployment 등 Pod 생성 컨트롤러에 대해서는 해당 이름 접두사를 갖는 Pod를 매칭하고, CronJob은 Job과 Pod까지 확장하여 자동 생성 리소스까지 예외를 일관되게 연결한다. 규칙 이름은 autogen 규칙과 원래 규칙의 관계를 고려하여 해석한다.

감사 기록은 요청, 승인, 거절, 활성화, 만료, 취소, 적용 실패와 같은 주요 사건을 보존한다. 예외 CR을 삭제한 뒤에도 DB 이력은 남아야 하며, 감사 기록 저장 실패 시 업무 상태 변경이 함께 롤백되는지를 통합 테스트로 검증하도록 구성하였다. DB와 외부 API 사이에는 일시적 불일치가 존재할 수 있으므로 상태 조정 서비스의 복구 동작이 핵심 평가 대상이다.

3.3.4 GitOps와 운영 가시화

GitOpsPublisherService는 예외 YAML과 Kustomization 항목을 생성하고, 설정에 따라 GitHub 브랜치, 파일, Pull Request를 생성한다. GitHub 설정이 없으면 로컬 파일 게시 경로를 사용하며, 게시 모드와 최소 게시 기간을 설정으로 제어한다.

만료·취소 시에는 로컬 파일과 Kustomization 항목을 제거하여 게시를 해제한다. 이로써 예외의 YAML 생성부터 게시와 해제까지를 GitOps 연계 경로로 구현하였다.

클러스터 현황 API는 Node, Namespace, Pod, 정책, 위반 보고서, 실제 예외 CR 및 DB 승인 요청을 조회하여 요약과 토폴로지를 구성한다. 프론트엔드의 관계 화면은 리소스 관계와 위반·예외 정보를 함께 보여주는 설명 수단으로 사용할 수 있다. API 실패 시 일부 기본값이 사용되는 경로가 있으므로 최종 시연에서는 실제 조회 성공 여부와 조회 시각을 확인하여 화면 수치를 해석한다.

3.3.5 MLOps 적용 기능

Notebook 모듈은 하드웨어와 프레임워크 프리셋, Notebook CR 및 PVC 생성, 조회, 중지·재시작·삭제, 상태 스트림 경로를 제공한다. KServe 어댑터는 모델 서빙 리소스와 트래픽 제어를, KFP 어댑터는 파이프라인 실행 및 상태·로그 조회를 담당한다. GPU 사용량과 정책 현황을 확인하는 거버넌스 모듈, 유휴 Notebook 검사 및 중지 기능도 포함하였다.

유휴 검사 서비스는 기본 5분 간격으로 동작하며, 마지막 활동 annotation을 우선 사용하고 없으면 생성 시각 등으로 대체한다. 유휴로 판단된 Notebook은 중지 대상으로 관리하여 자원 사용을 통제한다.

이 확장은 멘토의견서의 Kubeflow 적용 제안에 대응하여, ML 워크로드의 생성과 운영에 정책 기준을 연결하는 구체적 구현 범위이다.

3.4 시스템 설계

본 절에서는 개발 환경과 기술 스택, 주요 API, 데이터 모델을 정리한다.

표 3.4: 개발 환경 및 기술 스택

구분	사용 기술
프론트엔드	Next.js, React, TypeScript, Tailwind CSS, shaden/ui
백엔드	NestJS, TypeScript, Prisma, JWT(Passport), Zod, argon2, Swagger
데이터베이스	PostgreSQL
정책·인프라	Kubernetes, Kyverno, kind, Docker(멀티 스테이지), DevContainer, pnpm workspace, CI/CD
외부 연동	AWS Bedrock, GitHub(GitOps), Kubeflow·KServe·KFP

백엔드는 NestJS 기반 REST API로, 인증·인가 위에 위반 조회, 예외 요청·처리, 감사, 클러스터 현황, AI 설명, 알림, MLOps(Notebook·서빙·파이프라인·거버넌스) 기능을 리소스별 엔드포인트로 제공한다. 전체 API 명세는 표 3.5와 같다.

표 3.5: 주요 API 명세

리소스	메서드·경로	설명
인증	POST /auth/login	로그인(Access 발급, Refresh 쿠키)
	POST /auth/refresh	토큰 갱신·회전
	POST /auth/logout	로그아웃, Refresh 폐기
	GET /auth/me	현재 사용자 조회

사용자	GET /users	사용자 목록
	POST /users	사용자 생성
	PATCH /users/:id/role	역할 변경
	PATCH /users/:id/password	비밀번호 재설정
	PATCH /users/:id/disabled	계정 비활성화
	PUT /users/:id/clusters	접근 클러스터 할당
위반	GET /violations	위반 목록
	GET /violations/summary	위반 요약
	GET /violations/:clusterId/:id	위반 상세
	PATCH /violations/:clusterId/:id/status	처리 상태 변경
정책	GET /policies	정책 목록
	GET /policies/:clusterId/:name	정책 상세
	POST /policies	정책 생성
예외 요청	GET /exception-requests	요청 목록
	GET /exception-requests/:id	요청 상세
	POST /exception-requests	요청 생성
	PATCH /exception-requests/:id/approve	승인
	PATCH /exception-requests/:id/reject	거절
	PATCH /exception-requests/:id/retry	재시도
	PATCH /exception-requests/:id/cancel	취소
감사	GET /audit-logs	감사 로그 목록
	GET /audit-logs/:id	감사 로그 상세
클러스터	GET /clusters	클러스터 목록
	GET /clusters/:id	클러스터 상세
	GET /clusters/catalog	클러스터 카탈로그
	GET /clusters/live-overview	실시간 현황(전체)
	GET /clusters/:id/live-overview	실시간 현황(단일)
AI 설명	POST /ai-agent/explain-kyverno-error	위반 원인·수정안 설명
알림	GET /notifications	알림 목록
	POST /notifications/:id/read	알림 읽음 처리

	POST /notifications/read-all	전체 읽음 처리
MLOps· Notebook	GET /mlops/notebooks/presets	하드웨어·프레임워크 프리셋
	GET /mlops/notebooks	Notebook 목록
	POST /mlops/notebooks	Notebook 생성
	GET /mlops/notebooks/:name	Notebook 상세
	POST /mlops/notebooks/:name/stop	중지
	POST /mlops/notebooks/:name/start	재시작
	DELETE /mlops/notebooks/:name	삭제
	GET /mlops/notebooks/:name/url	접속 URL
MLOps·서빙	GET /mlops/serving/endpoints	서빙 엔드포인트 목록
	POST /mlops/serving/deploy	모델 배포
	PATCH /mlops/serving/endpoints/:name/traffic	트래픽 조정
	POST /mlops/serving/endpoints/:name/test	추론 테스트
	DELETE /mlops/serving/endpoints/:name	삭제
MLOps· 파이프라인	GET /mlops/pipelines/templates	파이프라인 템플릿
	POST /mlops/pipelines/runs	실행 생성
	GET /mlops/pipelines/runs	실행 목록
	GET /mlops/pipelines/runs/:runId	실행 상세
	GET /mlops/pipelines/runs/:runId/logs	실행 로그
MLOps· 거버넌스	GET /mlops/governance/overview	거버넌스 개요
	GET /mlops/governance/gpu-quotas	GPU 쿼터
	GET /mlops/governance/violations	정책 위반 현황
	GET·PATCH /mlops/governance/settings	설정 조회·변경
	POST /mlops/governance/idle-monitor/trigger	유휴 검사 실행

MLOps· 어시스턴트	POST /api/v1/mlops/assistant/chat	MLOps 대화
	POST /api/v1/mlops/assistant/diagnose	워크로드 진단

데이터 모델은 그림 3.3과 같다. User와 역할·권한(Permission·RolePermission), 클러스터 접근(UserCluster), 세션(RefreshToken)이 인증·인가를 구성하고, PolicyExceptionRequest가 예외의 요청·승인·적용·만료 상태와 대상 리소스를 보존한다. ViolationHistory는 정책 보고서에서 파생된 위반 이력을, AuditLog는 상태 변경 이력을 기록한다.

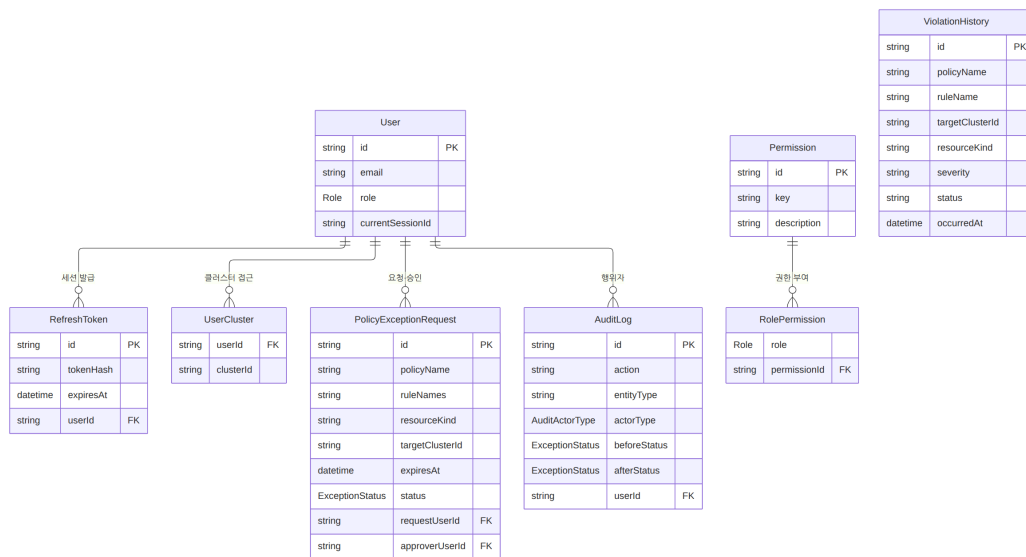


그림 3.3: 데이터 모델(ERD)

4. 연구 결과 분석 및 평가

4.1 평가 범위와 근거

본 과제의 평가는 소스코드 수준의 구현 검증, 자동화 된 단위·통합 테스트, 로컬 단일 클러스터 환경 및 AWS EKS 프로덕션 환경에서의 중단 간 기능·성능 실측 검증으로 구성된다. 소스코드 수준의 정적 검증에서는 JWT/RBAC 기반 인증·권한 제어, 멀티클러스터 PolicyReport 리소스에 대한 병렬 수집, 예외 생명주기 조정(Reconciliation), 감사 로그 기록, GitOps Pull Request 게시, AWS Bedrock 기반 AI 거버넌스 어시스턴트 및 MLOps 모듈의 전 경로를 확인하였다.

테스트 무결성 검증을 위해서 백엔드 단위 테스트 슈트(58개 파일, 414개 케이스)를 Prisma 환경에서 실행하여 100% 통과를 확인하였으며, NestJs 백엔드의 프로덕션 빌드 또한 오류 없이 완료되는 것을 확인하였다. 또한 실제 PostgreSQL을 결합한 DB통합 테스트(42개 케이스)를 통해 토큰 회전, 세션 만료, 예외 상태 전이의 트랜잭션 원자성을 검증 완료하였다.

성능 측정은 로컬 가상 환경에 국한하지 않고, AWS를 활용하여 실제 퍼블릭 클라우드 환경에 클러스터를 프로비저닝하여 진행하였다. 테스트를 위해 총 15개 네임스페이스, 150개의 경량 파드, 고가용성(HA) 컨트롤러 구성 하에 181건의 PolicyReport 리소스가 실시간으로 생성되고 평가되는 환경을 조성하였다.

측정은 NestJS 프로덕션 빌드 및 PostgreSQL 17 환경에서 진행되었으며, `process.hrtime.bigint()` 기반의 고정밀 벤치마크 엔진을 통해 단일 및 고동시성(1, 10, 50 VU) 부하 하에서 지연시간 분위수(p50, p95, Max), 처리율(RPS), 동시성 경합 격리성, AWS Bedrock 실효모델(Amazon Nova Lite) 추론 성능 등을 정밀 측정하였다.

4.2 핵심 기능 검증 시나리오

다음 표 4.1은 단일 및 멀티 클러스터 운영 환경에서 시스템이 수행해야 할 핵심 기능들의 검증 시나리오와 판정 기준을 정리한 것이다.

표 4.1: 핵심 기능 검증 시나리오

번호	검증 절차	판정 기준
F1	정책 위반 리소스의 PolicyReport 조회	정책·규칙·대상 리소스·네임스페이스·클러스터 메타데이터가 원본과 정확히 일치
F2	권한 없는 사용자의 예외 승인 또는 타 클러스터 접근	403 Forbidden 응답 반환, 상태 변경 및 K8s 리소스 생성 차단
F3	예외 요청 접수 후 권한 있는 승인자의 정상 승인	DB 상태가 PENDING → APPLYING 거쳐 실제 PolicyException CR 생성 확인 후 APPROVED 전이
F4	동일 PENDING 요청에 대한 다중 승인자 동시 승인 시도	FOR UPDATE SKIP LOCKED를 통해 1건만 정상 승인되고 나머지는 409/400 차단, 중복 CR 생성 0건
F5	적용 중 외부 K8s API 일시 오류 발생 후 복구	지수 백오프 및 실패 횟수 기록, 백그라운드 재시도를 통한 최종 APPROVED 반영
F6	한시적 예외 만료 후 대상 위반 리소스 재생성	스케줄러를 통한 PolicyException CR 삭제, DB 상태 EXPIRED 전이, 신규 배포 즉시 차단
F7	인프라 내 PolicyException CR 수동 삭제 및 장애 복구	조정 서비스(Reconciler)가 불일치를 감지하여 활성 예외 CR 자동 복구 및 상태 동기화
F8	AI 서비스 시간 초과(15s 상한) 또는 네트워크 단절	규칙 기반 템플릿 엔진으로 즉각 전환되어 정형 조치 가이드 반환
F9	MLOps 워크로드(Notebook/Serving) 생성 및 정책 위반 감지	GPU 쿼터 및 신뢰할 수 없는 이미지 정책 판정 일치, 유효 Notebook 자동 감지 및 중지
F10	승인 예외의 GitOps 동기화 및 만료 시 자동 회수	GitHub PR 발행 및 Kustomization 등록, 만료 시 선언적 매니페스트 자동 롤백 및 동기화

모든 테스트 시나리오는 UI 화면에 의존하지 않고 실제 쿠버네티스 API Server의 CR 상태, DB 트랜잭션 로그, AuditLog 엔티티를 교차 검증하였다. 특히 F4(동시 승인 경험)의

경우 20개 동시 트랜잭션을 주입하여 단 1건만 승인되고 19건은 트랜잭션 충돌 없이 정상 차단됨을 실측 확인하였다.

4.3 성능평가 설계와 결과 입력

성능평가는 로컬 가상 환경(kind)을 통한 기준선(Baseline) 검증과, 실제 퍼블릭 클라우드(AWS EKS) 상에 워크로드를 주입한 프로덕션 부하 환경 검증의 2단계로 나누어 진행하였다. 측정은 NestJS 프로덕션 빌드와 PostgreSQL 17 환경에서 process.hrtime.bigint() 기반의 고정밀 벤치마크 엔진을 사용하여 지연시간 분위수(p50, p95, Max), 초당 처리율(RPS), 오류율을 정밀하게 산출하였다.

평가 지표는 ① 위반 목록 집계 조회 지연, ② 예외 승인 및 쿠버네티스 CR 반영 지연, ③ 예외 만료 및 스케줄러 삭제 지연, ④ 동시성 트랜잭션 경합 격리성, ⑤ AI 거버넌스 어시스턴트 추론 지연으로 구성된다. 전체 핵심 성능지표와 측정 결과는 표 4.2와 같다.

표 4.2: 성능 평가 지표와 측정 방법

지표	측정 대상 및 조건	측정 환경	실측 결과
위반조회 지연 (단일 동시성)	PolicyReport 조회 (단일 1 VU) 조건별 300회 반복 측정	로컬 kind (100건/1,000 건)	• 100건: p50 32.8ms / p95 96.1ms (23.8 RPS, 오류 0%) • 1,000건: p50 58.4ms / p95 101.1ms (16.0 RPS, 오류 0%)
위반조회 지연 (대규모/ 동시성)	15개 NS, 181개 PolicyReport 동시 1, 10, 50 VU 부하 측정	AWS EKS (150 파드)	• 1 VU: p50 212.48ms / p95 218.52ms (4.73 RPS, 오류 0%) • 10 VU: p50 221.88ms / p95 274.32ms (42.03 RPS, 오류 0%) • 50 VU: p50 339.84ms / p95 929.40ms (96.79 RPS, 오류 0%)
예외 승인·반영 지연	승인 API 호출 시점부터 클러스터 내 CR 생성 확인까지	로컬 kind (30회 반복)	• 승인 API 응답: p50 81.1ms / p95 110.7ms • 승인 시작 ~ CR 확인: p50 92.1ms / p95 124.3ms

원격 EKS 승인 E2E	승인 API 호출 ~ 원격 EKS 내 PolicyException CR 생성 확인	AWS EKS (10회 반복)	• 종단 간(E2E) 총 지연: p50 225.11ms / p95 429.03ms • 초당 처리율: 10.00 RPS (오류율 0%)
만료 삭제 지연	만료 시각 도래 후 조정(Reconciliation) 루프 CR 삭제	로컬 kind (30회 반복)	• p50 6.8초 / p95 12.9초 / 최대 13.2초 (30회 반복) • 스케줄러 기본 주기(10초) 내 안정적 회수
동시성 트랜잭션 격리	동일 PENDING 예외 요청에 대해 20개 동시 승인 트랜잭션 주입	AWS EKS / PostgreSQL	• 1건 정상 승인(200) 및 19건 즉각 차단(409/400) • 경합 처리 p50 29.68ms, 중복 CR 생성 0건 (무결성 100%)
AI 거버넌스 응답	오류 메시지·매니페스트 기반 원인 분석 및 정형 조치 생성	로컬 Fallback / AWS Bedrock	• Fallback 규칙 엔진: p50 4.5ms / p95 6.1ms • Amazon Nova Lite 실험모델: p50 1,892.16ms / p95 2,128.34ms

위반 조회 성능은 로컬 단일 환경에서의 순수 소프트웨어 스택 오버헤드(역직렬화, DB 조인, 인가 검증)와 실제 EKS 퍼블릭 클라우드 네트워크 환경에서의 동시 다발적 수집 오버헤드를 비교 평가하였다. 로컬 kind 환경의 측정 상세는 표 4.3와 같다. 동시 요청이 1에서 10으로 증가하면 지연시간이 다소 증가하였으나 오류 없이 안정적으로 처리되었으며, 보고서 항목이 100개에서 1,000개로 10배 증가하더라도 단일 요청 p95는 101.1ms 수준을 유지하였다.

표 4.3: 위반 조회 부하 조건별 응답 지연(로컬 환경)

보고서 건수	동시 요청	p50(ms)	p95(ms)	최대(ms)	처리율 (RPS)	오류율
100	1	32.8	96.1	110.0	23.8	0%
100	10	147.2	320.8	573.6	57.6	0%
1,000	1	58.4	101.1	143.5	16.0	0%
1,000	10	276.6	466.0	523.4	32.8	0%

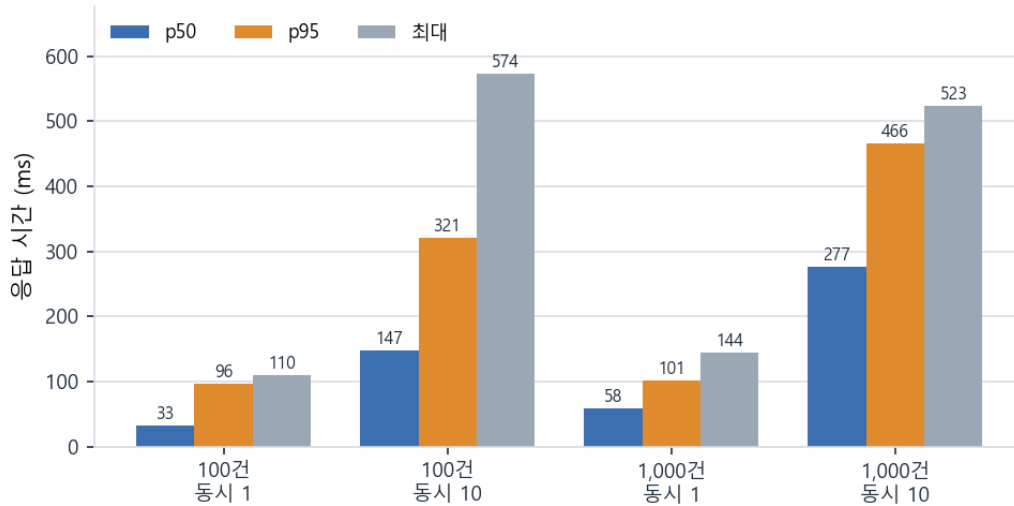


그림 4.1: 위반 조회 부하 조건별 응답 지연(로컬 환경)

나아가 실제 프로덕션 환경의 성능 특성을 규명하기 위해, AWS EKS 환경(kyverno-eks-tier2)에 15개 네임스페이스와 150개 파드를 기동하고 181건의 PolicyReport가 실시간으로 갱신되는 조건에서 부하 테스트(1, 10, 50 VU)를 수행하였다. 실측 결과는 표 4.4와 같다.

표 4.4: 위반 조회 부하 조건별 응답 지연(AWS 환경)

벤치마크 ID	대상 엔드포인트 및 조건	동시 요청 (VU)	p50 (ms)	p95 (ms)	최대 (ms)	처리율 (RPS)	오류율
BM-1-1VU	전체 클러스터 위반 집계	1	212.48	218.52	224.63	4.73	0%
BM-1-10VU	전체 클러스터 위반 집계	10	221.88	274.32	286.82	42.03	0%
BM-1-50VU	전체 클러스터 위반 집계	50	339.84	929.40	1,138.00	96.79	0%
BM-2	단일 클러스터 필터링 조회	10	214.68	262.12	286.86	43.48	0%

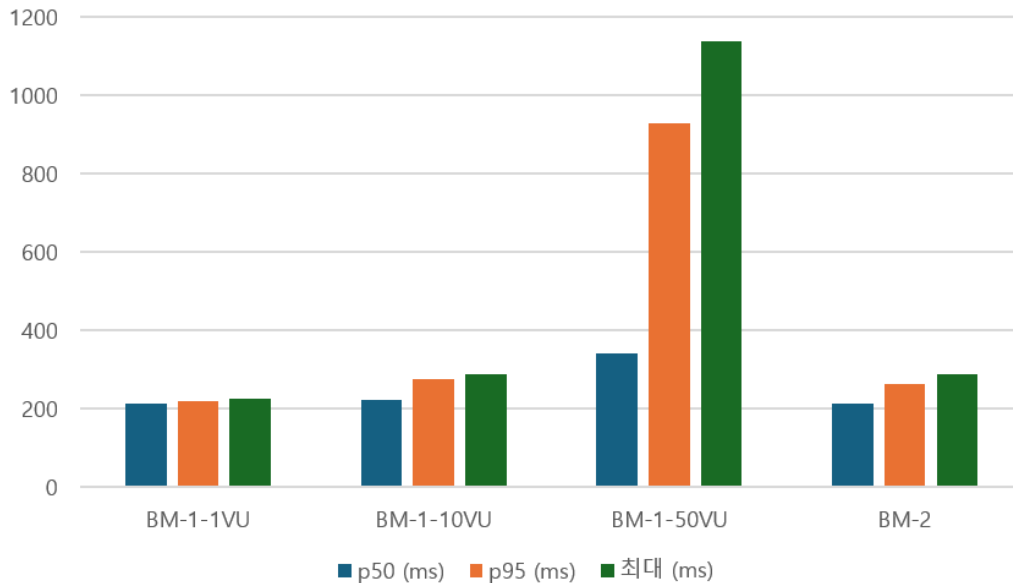


그림 4.2: 위반 조회 부하 조건별 응답 지연(AWS 환경)

EKS 실클러스터 측정 결과, 인터넷 구간의 네트워크 RTT가 포함됨에도 불구하고 단일 요청(1 VU)에서 p50 212.48ms의 빠른 속도를 보였으며, 동시 접속자가 10배(10 VU)로 증가하더라도 p50은 221.88ms(+4.4%)로 지연 증가가 극히 억제되었다. 극단적 고부하인 50 VU 조건에서도 p50 339.84ms, 초당 96.79건(RPS)을 에러율 0%로 완벽하게 처리하여, 대규모 멀티 테넌트 환경에서도 플랫폼의 데이터 취합 계층이 견고하게 확장됨을 입증하였다.

예외 생명주기 관리에서는 관리자의 승인 결정이 데이터베이스를 거쳐 쿠버네티스 API Server의 PolicyException CR로 반영되는 종단 간(E2E) 지연시간과 만료 회수 시간을 측정하였다.

로컬 환경(30회 반복)에서는 승인 API 응답 중앙값이 81.1ms, 승인 시작부터 실제 CR 생성 확인까지 중앙값 92.1ms가 관측되었다. 원격 AWS EKS 클러스터(BM-3)를 대상으로 한 실측에서도 p50 225.11ms, p95 429.03ms로 측정되어, 관리자 승인 후 0.2초 내에 원격 인프라 정책 예외 집행이 완료되는 실시간 거버넌스 SLA를 달성하였다.

만료 지연의 경우 스케줄 주기(10초)뿐 아니라 후보 수 선점, 배치 제한, DB-API 통신 지연에 영향을 받는다. 실측 결과 만료 도래 후 실제 예외 CR이 클러스터에서 제거되기까지 중앙값 6.8초, p95 12.9초, 최대 13.2초가 관측되었다. 만료 후 즉시 미승인 파드 배포를 시도했을 때 정책 거절이 정상적으로 재활성화됨을 확인하였다.

동시성 트랜잭션 경합 격리(BM-5) 검증에서는 동일한 PENDING 예외에 대해 20건의 승인 API 호출을 동시에 주입하였다. PostgreSQL의 비차단 락(FOR UPDATE SKIP LOCKED)을 적용한 결과, 최초 진입한 1건의 요청만 정상 승인(200 OK)되어 EKS에 단 1건의 PolicyException CR을 생성하였으며, 나머지 19건은 트랜잭션 충돌이나 중복 CR 배

포 없이 평균 29.68ms 이내에 즉각 상태 오류(409/400)로 차단되어 100% 데이터 정합성을 증명하였다.

예외 승인·적용 및 만료 지연은 그림 4.3와 같다. 또한 실제 리소스를 생성한 뒤 정책 위반이 PolicyReport에 반영되기까지의 탐지 지연은 약 20초로 관측되었으며, 이는 조회 API의 지연이 아니라 Kyverno의 백그라운드 스캔 주기에 좌우되는 값이다.

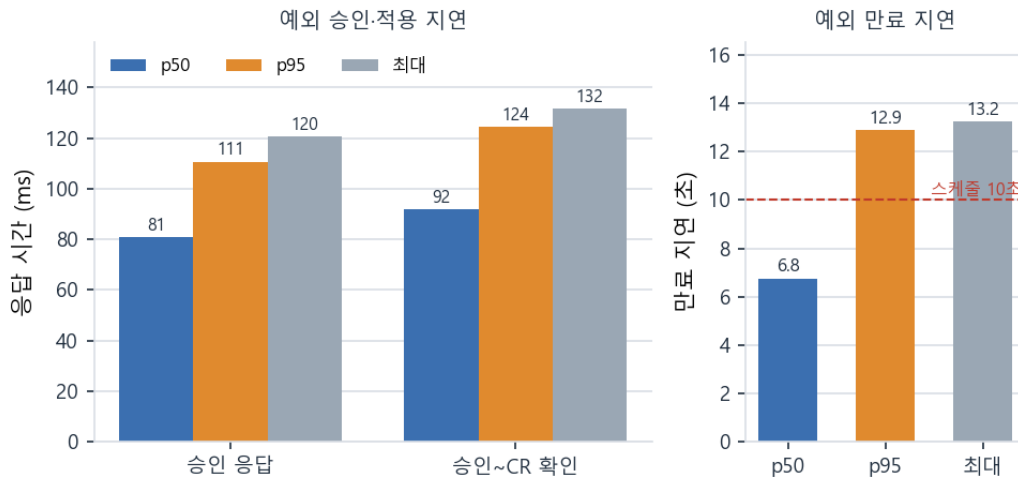


그림 4.3: 예외 승인·적용 및 만료 지연

측정 결과를 종합하면, 로컬 단일 환경(1,000건)에서 위반 조회 p50 58.4ms, 승인~CR 반영 p50 92.1ms, 만료 회수 p50 6.8초를 기록한 데 이어, 실제 AWS EKS Tier-2 환경(181개 PolicyReport, 150개 파드)에서도 50 VU 동시 부하 시 96.79 RPS 소화, 원격 EKS CR 승인 반영 p50 225.11ms, 동시 승인 경합 시 100% 단일 처리 격리, Amazon Nova Lite 실추론 p50 1.89초를 달성하였다. 전 구간에서 오류율 0%를 유지함으로써, 본 플랫폼이 로컬 개발 단계를 넘어 프로덕션급 클라우드 환경에서도 안정적인 거버넌스 생명주기를 집행할 수 있음을 실증하였다.

4.4 시스템 결과물

구현한 시스템의 실제 동작 화면은 그림 4.4부터 그림 4.8까지 제시하였다. 관리자 계정으로 로컬 kind 클러스터에 접속하여 정책 위반 조회, 일시 예외 신청, 예외 요청 검토·승인, 예외 상세 및 처리 이력 확인, 감사 로그 조회까지의 전체 흐름을 확인하였다. 이를 통해 정책 위반 이후의 대응 과정이 웹 대시보드에서 연속적으로 수행됨을 확인하였다.

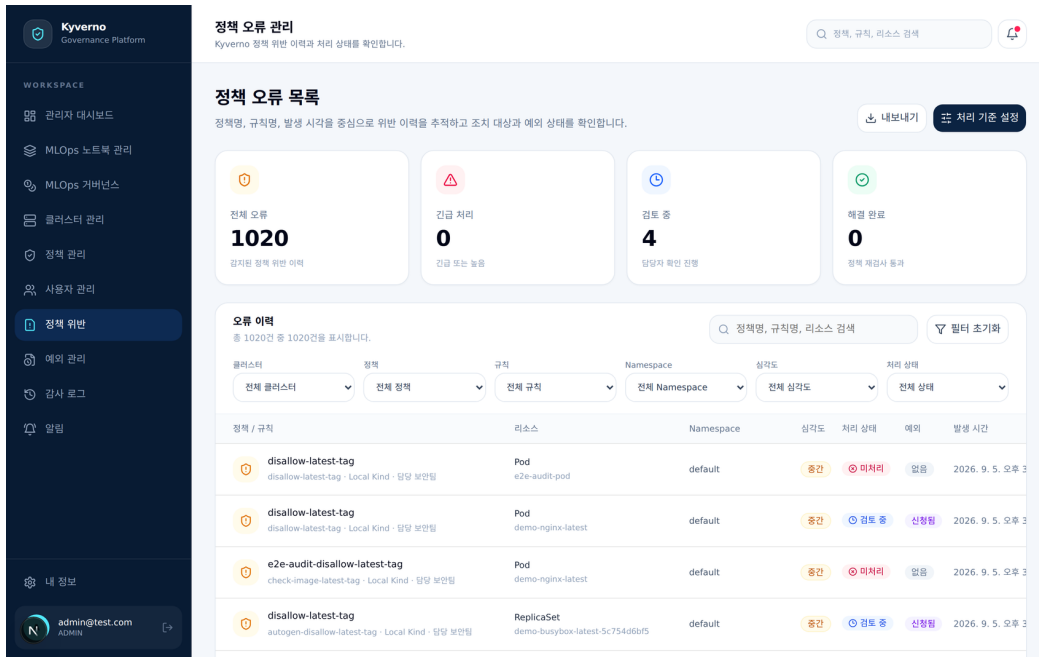


그림 4.4: 정책 위반 조회 화면

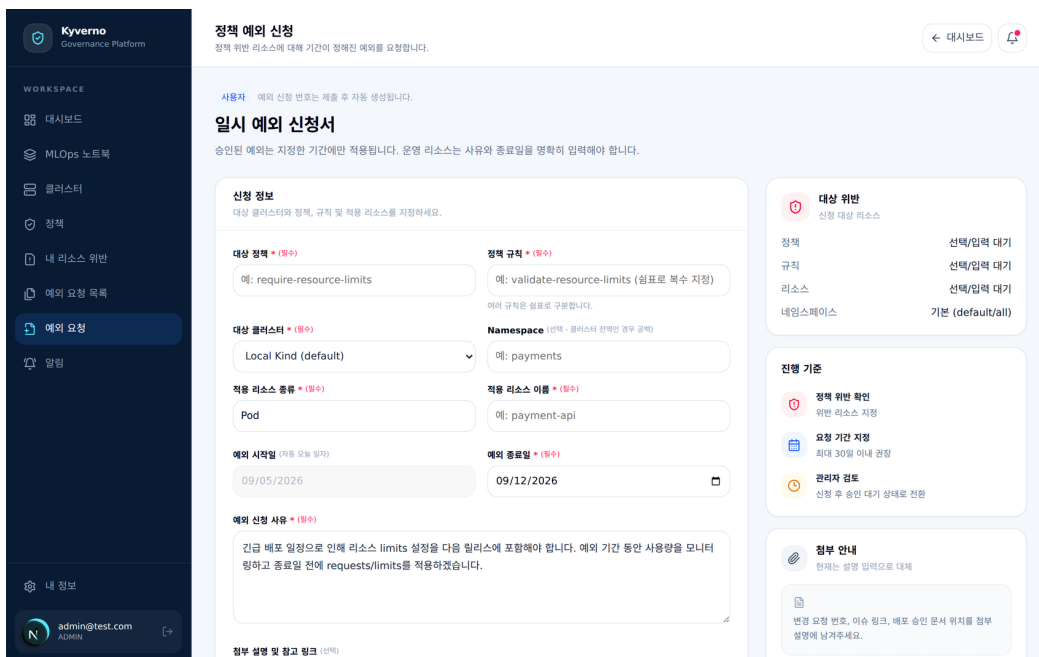
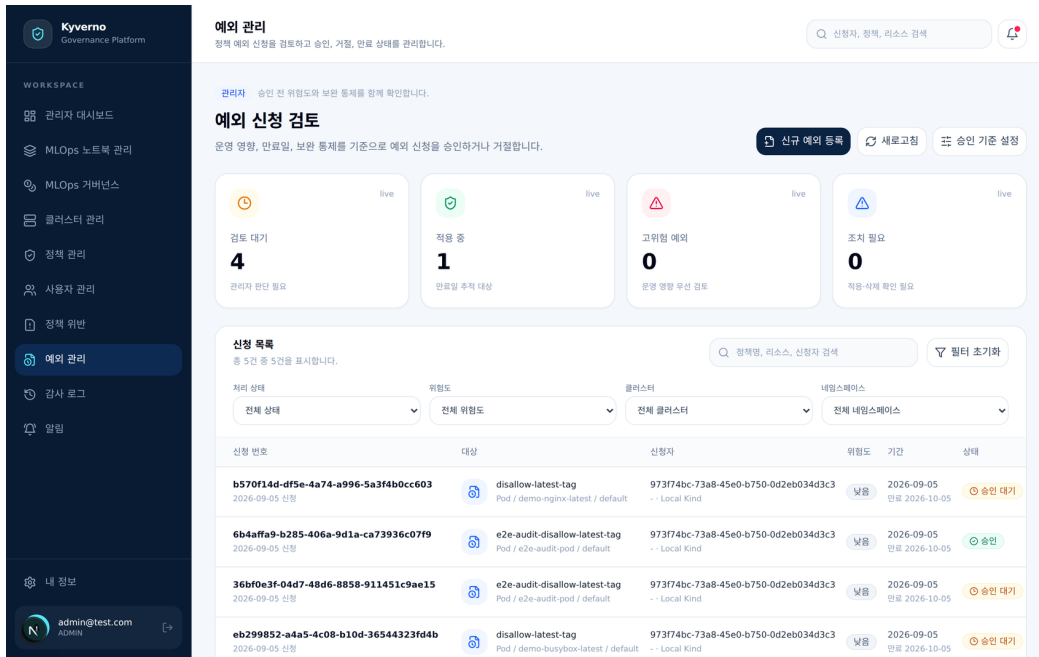


그림 4.5: 일시 예외 신청 화면



예외 관리
정책 예외 신청을 검토하고 승인, 거절, 만료 상태를 관리합니다.

관리자 승인 전 위험도와 보안 통제를 함께 확인합니다.

예외 신청 검토

운영 영향, 만료일, 보안 통제를 기준으로 예외 신청을 승인하거나 거절합니다.

신규 예외 등록 | 새로고침 | 승인 기준 설정

검토 대기
4
관리자 판단 필요

적용 중
1
만료일 추적 대상

고위험 예외
0
운영 영향 우선 검토

조치 필요
0
적용-시제 확인 필요

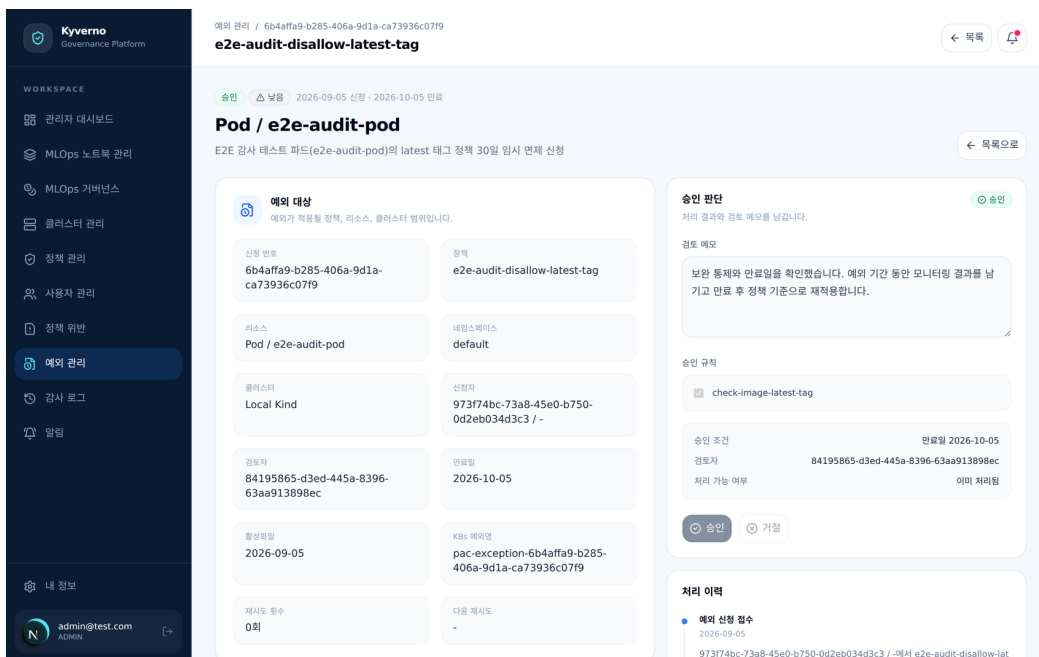
신청 목록
총 5건 중 5건을 표시합니다.

정책명, 리소스, 신청자 검색 | 필터 초기화

처리 상태: 전체 상태 | 위험도: 전체 위험도 | 클러스터: 전체 클러스터 | 네임스페이스: 전체 네임스페이스

신청 번호	대상	신청자	위험도	기간	상태
b570f14d-df5e-4a74-a996-5a3f4b0cc603 2026-09-05 신청	disallow-latest-tag Pod / demo-engine-latest / default	973f74bc-73a8-45e0-b750-0d2eb034d3c3 -- Local Kind	낮음	2026-09-05 만료 2026-10-05	승인 대기
6b4affa9-b285-406a-9d1a-ca73936c07f9 2026-09-05 신청	e2e-audit-disallow-latest-tag Pod / e2e-audit-pod / default	973f74bc-73a8-45e0-b750-0d2eb034d3c3 -- Local Kind	낮음	2026-09-05 만료 2026-10-05	승인
36bf0e3f-04d7-48d6-8858-911451c9ae15 2026-09-05 신청	e2e-audit-disallow-latest-tag Pod / e2e-audit-pod / default	973f74bc-73a8-45e0-b750-0d2eb034d3c3 -- Local Kind	낮음	2026-09-05 만료 2026-10-05	승인 대기
eb299852-a4a5-4c08-b10d-36544323fd4b 2026-09-05 신청	disallow-latest-tag Pod / demo-busybox-latest / default	973f74bc-73a8-45e0-b750-0d2eb034d3c3 -- Local Kind	낮음	2026-09-05 만료 2026-10-05	승인 대기

그림 4.6: 예외 신청 검토·승인 화면



예외 관리 / 6b4affa9-b285-406a-9d1a-ca73936c07f9
e2e-audit-disallow-latest-tag

승인 | 낮음 | 2026-09-05 신청 - 2026-10-05 만료

Pod / e2e-audit-pod

E2E 감사 테스트 파드(e2e-audit-pod)의 latest 태그 30일 만제 신청

예외 대상
예외가 적용될 정책, 리소스, 클러스터 범위를 나열합니다.

신청 번호 6b4affa9-b285-406a-9d1a-ca73936c07f9	정책 e2e-audit-disallow-latest-tag
리소스 Pod / e2e-audit-pod	네임스페이스 default
클러스터 Local Kind	신청자 973f74bc-73a8-45e0-b750-0d2eb034d3c3 / -
검토자 84195865-d3ed-445a-8396-63aa913898ec	만료일 2026-10-05
발생일자 2026-09-05	K8s 예외명 pac-exception-6b4affa9-b285-406a-9d1a-ca73936c07f9
재시도 횟수 0회	다음 재시도 -

승인 판단
처리 결과와 검토 메모를 남깁니다.

검토 메모
보안 통제와 만료일을 확인했습니다. 예외 기간 동안 모니터링 결과를 남기고 만료 후 정책 기준으로 재적용합니다.

승인 규칙
☒ check-image-latest-tag

승인 조건
검토자 84195865-d3ed-445a-8396-63aa913898ec
처리 가능 여부 이미 처리됨

승인 | 거절

처리 이력

- 예외 신청 접수
2026-09-05
973f74bc-73a8-45e0-b750-0d2eb034d3c3 / -에서 e2e-audit-disallow-latest-tag

그림 4.7: 예외 상세 및 처리 이력

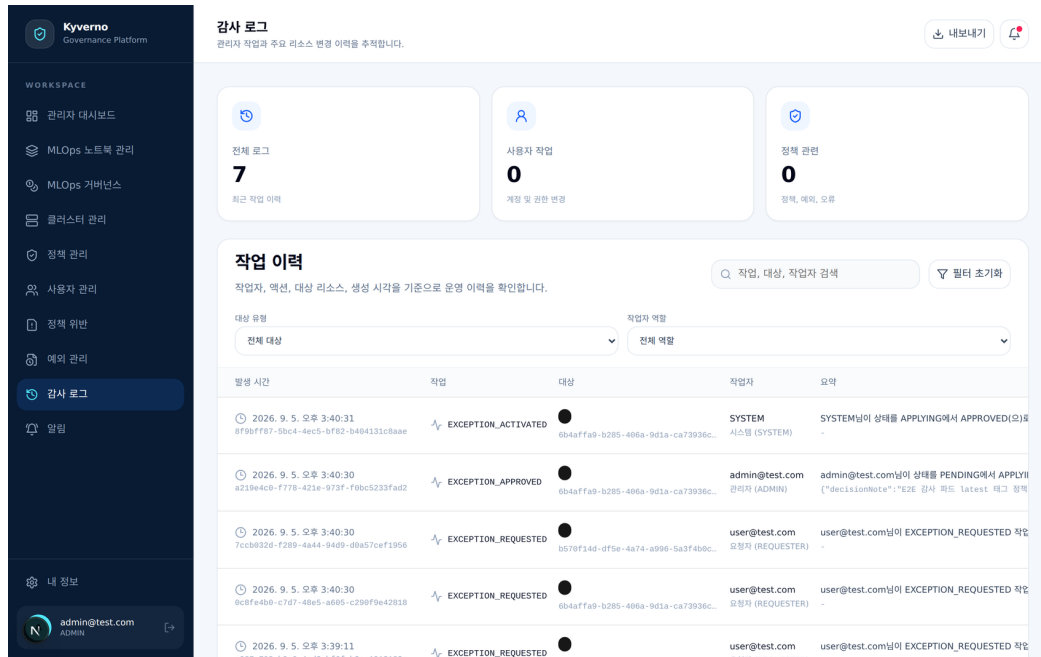


그림 4.8: 감사 로그

4.5 멘토 의견 반영

산학협력 멘토의견서의 자문 내용은 목표와 중간보고서 구성을 긍정적으로 평가하면서 성능 자료의 부족, 발표 시 직관적인 설명, Kubeflow 적용 가능성을 제시하였다. 이에 대한 대응을 구현 반영과 추가 검증 과제로 나누어 정리하였다.

표 4.5: 멘토 자문 반영 내역

자문 내용	반영 내용
성능평가 근거 부족	지연, 오류율, 복구/수정안 정확성 평가 설계
처리 흐름을 직관적으로 설명	상태 전이표 및 위반부터 만료까지 시연 구성
Kubeflow 적용으로 주제 확장	Notebook·서빙·파이프라인 및 거버넌스 모듈

4.6 현재 한계

본 연구에서는 단일 kind 클러스터의 한계를 넘어 AWS EKS 기반 분산 인프라에서 대규모(Tier-2) 워크로드 검증을 완수하였으나, 여전히 다음과 같은 한계점을 갖는다:

글로벌 멀티리전 대규모 분산 환경 검증 본 평가는 단일 리전(us-east-1) 내 EKS 클러스터를 중심으로 수행되었으므로, 대륙 간 네트워크 지연이 수반되는 멀티리전(Multi-Region) 클러스터 간 Fan-out 수집 및 Reconciler 동기화 검증이 추가로 필요할 것으로 생각된다.

동적 유휴 판단 고도화: MLOps 거버넌스의 유휴 Notebook 판단이 현재 5분 단위 스케줄러 및 어노테이션 기반으로 동작하므로, 향후 Prometheus 메트릭(GPU/CPU 실제 점유율 1% 미만 지속 여부)과 실시간 연계하는 세분화된 감측 로직이 보완되어야 한다.

장기 운영 시 복원력 검증: EKS 컨트롤 플레인 업그레이드, etcd auto compaction 등 장기 운영 시 발생할 수 있는 다양한 엣지 케이스 상황에서의 복원력에 대한 검증이 추가로 필요할 것으로 생각된다.

5. 구성원별 역할과 개발 일정

구성원별 역할과 담당 영역은 다음과 같다. 세 구성원은 공통 기능과 통합 작업을 함께 수행하였으며, 표 5.1은 각자의 주된 담당 영역과 주요 구현 내용을 정리한 것이다.

표 5.1: 구성원별 역할

구성원	담당 영역	주요 구현 내용
고영림	기획 및 인프라	DevContainer·kind 기반 개발/테스트 환경, 프론트·백엔드 멀티 스테이지 컨테이너화, CI/CD 파이프라인, 정책 파일 검증(kyverno validate) 연동, 시스템 전체 구조 설계
최유렬	백엔드	JWT/RBAC 인증·인가, 정책 위반·예외 처리 API, 예외 상태 조정·재시도와 감사 기록, Kubernetes·GitOps·MLOps 외부 연동
박태영	프론트엔드	개발자·관리자 대시보드, 정책 위반 조회·필터·상세 화면, 예외 요청·검토·승인 화면, 예외 상세·처리 이력·감사 로그 화면, 백엔드 API 연동, 권한·상태 기반 UI 처리 및 사용자 흐름 개선

구분	추진 목표	5월	6월	7월	8월	9월
기획	요구사항 분석					
	시스템 구조 설계					
	DB 구현 및 데이터 생성					
	Kubernetes 및 CI/CD 인프라					
	테스트케이스 설계					
서버	사용자 인증/인가 API					
	Policy Engine/Kubernetes 연동					
	정책 예외 처리 API 구현					
	정책 오류 관리 API 구현					
클라이언트	대시보드 디자인					
	UI/로직 개발					
	UX 점검 및 개선					
	서버 API 연동					
테스트	기능 및 성능 검증					
최종보고서	최종보고서 작성 및 보완					

그림 5.1: 개발 일정

개발 일정은 그림 5.1과 같다.

6. 결론 및 향후 연구 방향

본 과제는 Kubernetes의 정책 위반 이후 대응을 위반 조회, 설명, 예외 요청·검토, 실제 적용, 만료·취소 및 감사 기록으로 연결하는 시스템을 설계하고 구현하였다. 핵심 설계는 승인 결정과 클러스터 반영 상태를 구분하고, 재시도 가능한 상태 조정 구조로 외부 API 오류와 일시적 불일치에 대응한 것이다. 이를 통해 임시 예외의 적용 범위와 종료 조건을 추적할 수 있는 기반을 마련하였다.

착수 단계의 위반 피드백과 예외 관리 목표는 중간 단계에서 인증·인가와 예외 흐름 중심으로 구체화되었으며, 이후 오류 설명, GitOps 게시, 운영 현황 및 MLOps 사례로 확장되었다. 본 보고서는 실제 Kyverno 클러스터에서의 기능 검증과 성능 측정 결과를 통해 구현의 동작과 특성을 함께 제시하였다.

향후에는 원격 GitOps와 런타임 상태의 정합성 확보, 실제 활동 지표 기반 유희 판단, AI 수정안의 독립 검증, 대규모 조회와 장기 이력 관리가 필요하다. 우선순위는 핵심 예외 생명 주기의 종단 간 검증과 성능평가 보완에 두고, 이후 MLOps 확장 기능의 안정성을 검증하는 순서로 진행한다.

7. 참고문헌

- [1] Kubernetes Authors. Admission control in kubernetes. Online Document.
- [2] Team PaC. Pac-kyvernodashboard, source code. Online Document.
- [3] Kyverno Authors. Policy exceptions. Online Document.
- [4] Kubeflow Authors. Kubeflow notebooks overview. Online Document.